

Module 3: Data Warehousing & OLAP Cubes

Star/Snowflake Schemas, Data Cube Computation & 5 OLAP Operations

Module III: Data Warehousing & OLAP Technology — 9 Topics Syllabus

- | | |
|---|--|
| 1. Basic Concepts of Data Warehouse (4 Pillars, OLTP vs OLAP) | 2. Data Warehouse Modeling (Fact, Dimensions, Measures) |
| 3. Data Cube & Lattice of Cuboids Formulation | 4. OLAP Operations (Roll-up, Drill-down, Slice, Dice, Pivot) |
| 5. Schema Design & SQL DDL (Star, Snowflake, Galaxy) | 6. Data Warehouse Implementation (ROLAP, MOLAP, HOLAP, Indexing) |
| 7. Generalization by Attribute-Oriented Induction (AOI Trace) | 8. Data Cube Computation (Multiway Array, BUC, Star-Cubing) |
| 9. Materialization Strategies (Full, Partial, Greedy Algorithm) | |

Topic 1: Basic Concepts of Data Warehouse

According to William H. Inmon, the acknowledged father of the data warehouse, a **Data Warehouse** is defined as:

William H. Inmon's Definitive Definition

*"A data warehouse is a **subject-oriented**, **integrated**, **time-variant**, and **non-volatile** collection of data in support of management's decision-making process."*

The Four Foundational Pillars:

- **1. Subject-Oriented:** Organized around major business subjects (e.g., Customer, Product, Supplier, Sales) rather than day-to-day transactional applications (e.g., Order Entry, Invoicing). Focuses on modeling and analysis for decision makers.
- **2. Integrated:** Built by unifying multiple heterogeneous data sources (RDBMS, flat files, online logs). Discrepancies in naming conventions, measurement scales, and encoding formats are resolved and cleansed during ETL (Extract-Transform-Load).
- **3. Time-Variant:** Data is stored as historical snapshots covering 5 to 10 years. Every key structure in a data warehouse explicitly contains an element of time (e.g., `Day\_ID`, `Month\_ID`, `Fiscal\_Year`).
- **4. Non-Volatile:** A data warehouse is physically separate from operational systems. Operational updates, deletions, and rollbacks do not occur. Only two data operations exist: **initial data loading** and **read-only analytical access**.

10–Point Technical Comparison: OLTP vs. OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Primary Users	Clerks, DBAs, Operational Engineers	Knowledge Workers, Business Analysts, Executives
System Function	Day-to-day transaction processing	Decision support, strategic trend analytics
DB Design	Highly normalized (3NF / BCNF)	Denormalized Multidimensional (Star / Snowflake)
Data Orientation	Application-oriented (Orders, Invoices)	Subject-oriented (Sales, Revenue, Customer)
Data Content	Current, real-time snapshot; detailed	Historical snapshots (5–10 yrs); summarized & detailed
Access Frequency	High frequency, short atomic read/write	Moderate frequency, complex long read-only queries
Unit of Work	Short simple transactions (INSERT/UPDATE)	Complex aggregation queries over millions of rows
Records Accessed	Tens to hundreds of records per query	Millions of records aggregated per query
Database Size	Gigabytes to tens of Gigabytes	Terabytes to Petabytes
Optimization Focus	High transaction throughput & zero lock contention	High query throughput & ultra-fast response time

Topic 2 & 5: Multidimensional Modeling & Warehouse Schemas

The data warehouse is modeled around a **Multidimensional Data Model** composed of:

- **Fact Table:** Contains numerical **Measures** (e.g., `dollars\_sold`, `units\_shipped`) and foreign keys referencing dimension tables.
 - *Additive Measures:* Can be summed along all dimensions (e.g., `dollars\_sold`).
 - *Semi-Additive Measures:* Can be summed along some dimensions but not others (e.g., `account\_balance` can be summed across branches, but not across time).
 - *Non-Additive Measures:* Cannot be summed along any dimension (e.g., `unit\_price`, ratios).
- **Dimension Tables:** Describe the context and perspectives of the facts (e.g., `Item`, `Time`, `Branch`, `Location`).

Schema Model	Design Architecture & Normalization Level	Key Performance & Query Tradeoffs
1. Star Schema	Central fact table surrounded by non-normalized (denormalized) dimension tables forming a star pattern.	Fast join queries; simple SQL navigation; slight data redundancy in dimension tables.
2. Snowflake Schema	Dimension tables are normalized into hierarchical sub-tables (e.g., `Item` splits into `Item`, `Subcategory`, `Category`).	Eliminates dimensional redundancy; reduces storage space; complex multi-table SQL joins reduce analytical query performance.
3. Fact Constellation (Galaxy Schema)	Multiple fact tables sharing common conformed dimension tables (e.g., `Sales_Fact` and `Shipping_Fact` both sharing `Time` and `Item`).	Sophisticated enterprise-wide model; supports cross-functional enterprise reporting.

Sample SQL DDL for Star Schema:

```
-- Fact Table DDL for AllElectronics Sales
CREATE TABLE Sales_Fact (
  time_key INT,
  item_key INT,
  branch_key INT,
  location_key INT,
  units_sold INT,
  dollars_sold DECIMAL(12, 2),
  avg_sales DECIMAL(10, 2),
  PRIMARY KEY (time_key, item_key, branch_key, location_key),
  FOREIGN KEY (time_key) REFERENCES Time_Dim(time_key),
  FOREIGN KEY (item_key) REFERENCES Item_Dim(item_key),
  FOREIGN KEY (branch_key) REFERENCES Branch_Dim(branch_key),
  FOREIGN KEY (location_key) REFERENCES Location_Dim(location_key)
);
```

Topic 3: The Data Cube & Lattice of Cuboids

A multidimensional data model represents data in the form of a **Data Cube**. In an n -dimensional space, data can be aggregated across all possible 2^n dimensional combinations, forming a hierarchical **Lattice of Cuboids**.

- **Base Cuboid (0-D Aggregation):** The most detailed, granular level containing all n dimensions.
- **Apex Cuboid (n -D Aggregation / All-Cuboid):** The highest level of generalization containing the grand total measure across all dimensions.
- **Total Number of Cuboids:** For a warehouse with n dimensions, where dimension i has a concept hierarchy with L_i levels:

$$T = \prod_{i=1}^n (L_i + 1)$$

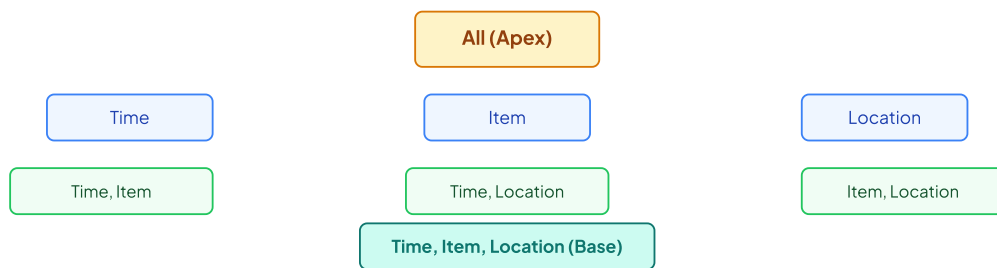


FIGURE 3.1: LATTICE OF CUBOIDS FOR A 3-DIMENSIONAL DATA CUBE ($2^3 = 8$ CUBOIDS)

Topic 4: Core OLAP Operations (The 5 Movements)

OLAP Operation	Formal Definition & Dimensional Manipulation	Concrete Business Example
1. Roll-Up (Drill-Up)	Performs aggregation along a dimension by climbing up a concept hierarchy or by dimension reduction.	Aggregating daily sales to quarterly sales; dropping the `Location` dimension to view total national revenue.
2. Drill-Down (Roll-Down)	Reverse of roll-up; navigates from less detailed data to more detailed data by stepping down a concept hierarchy or introducing a new dimension.	Expanding quarterly revenue by state down to monthly sales by individual retail store.
3. Slice	Performs a selection on a single dimension of the data cube, yielding a 2D sub-table.	Selecting `Time = "Q1 2026"` to inspect all item sales across locations for that quarter.
4. Dice	Defines a sub-cube by performing a selection on two or more dimensions simultaneously.	Selecting `Time = "Q1 2026"` AND `Location = "India"` AND `Item = "Laptop"`.
5. Pivot (Rotate)	Rotates the data axes in multidimensional space to provide an alternative visual presentation.	Swapping rows (Locations) and columns (Item Categories) in a reporting spreadsheet.

Topic 6: Data Warehouse Server Architectures (ROLAP vs. MOLAP vs. HOLAP)

Server Architecture	Internal Storage & Implementation	Key Tradeoffs
1. ROLAP (Relational OLAP)	Stores multidimensional data directly inside relational tables using Star/Snowflake schemas; uses specialized SQL middleware to translate OLAP queries into SQL joins.	Highly scalable to petabyte volumes; leverages existing RDBMS; query performance can be slower on complex aggregations.
2. MOLAP (Multidimensional OLAP)	Stores data directly in multidimensional array storage structures; pre-calculates and materializes cuboids.	Blazing fast query response times; optimal for dense cubes; suffers storage explosion on sparse data cubes.
3. HOLAP (Hybrid OLAP)	Stores detailed granular base data in relational tables (ROLAP) and stores high-level aggregated summary cuboids in multidimensional arrays (MOLAP).	Combines the massive storage capacity of ROLAP with the high query performance of MOLAP.

Topic 7: Generalization by Attribute-Oriented Induction (AOI)

Attribute-Oriented Induction (AOI) is a target-class concept generalization technique that extracts summary rules from relational tables using concept hierarchies without expert manual intervention.

The AOI Algorithm: Two Core Generalization Operators

1. **Attribute Removal:** If an attribute has a large number of distinct values and no generalization concept hierarchy is available (e.g., `Social\_Security\_Number`, `Employee\_ID`), the attribute is removed from the generalized relation.
2. **Attribute Generalization:** If an attribute has an associated concept hierarchy and the number of distinct values exceeds a user-specified threshold, replace lower-level values with higher-level concept concepts until the attribute threshold is satisfied.
3. **Tuple Aggregation:** Merge identical generalized tuples and accumulate count/sum measures.

Step-by-Step Solved Problem: AOI on University Graduate Database

Initial Relation: $T = (\text{Name}, \text{Gender}, \text{Major}, \text{Birth_Place}, \text{GPA})$.

1. `Name` has 5,000 distinct values and no concept hierarchy $\implies$ **Attribute Removal**.
2. `Gender` has 2 distinct values $\leq \text{Threshold}(3)$ $\implies$ **Retain**.
3. `Major` has hierarchy (CSE, ECE, ME $\rightarrow$ Engineering; Physics, Chem $\rightarrow$ Science) $\implies$ **Generalize to Department**.
4. `Birth\_Place` has hierarchy (City $\rightarrow$ Province/State $\rightarrow$ Country) $\implies$ **Generalize to Country**.
5. `GPA` continuous numeric $\implies$ **Discretize to {Excellent, Good, Fair}**.
6. **Final Generalized Relation:** (Gender, Department, Country, GPA_Class, Count%).

Topic 8 – 9: Data Cube Computation & Materialization Strategies

1. Multiway Array Aggregation for Full Cube Computation:

Multiway Array Aggregation computes full data cubes by partitioning the multidimensional array into chunks that fit in main memory. By ordering the dimension aggregations such that smallest dimensions are kept in memory longest ($A \rightarrow B \rightarrow C$), the full cube is computed in a single pass over the base data.

2. Bottom-Up Computation (BUC):

BUC explores the lattice of cuboids bottom-up (starting from 1-D cuboids to n -D base cuboids). It exploits the **Iceberg Cube** pruning condition: if a cuboid cell has `count` < `min_sup`, BUC immediately prunes all descendant specialized cuboid cells, avoiding expensive aggregate queries.

3. Star-Cubing Algorithm:

Star-Cubing integrates the advantages of Star-Trees and Bottom-Up computation. It builds a shared prefix tree for the base cuboid and simultaneously compresses and computes shared sub-cuboid aggregates.

4. Materialization Tradeoffs:

- **Full Materialization:** Pre-computes all 2^n cuboids. Instantaneous query execution, but requires exponential disk space.
- **No Materialization:** Zero extra storage, but computing aggregates on-the-fly from terabyte base tables yields unacceptable query latencies.
- **Partial Materialization:** Selects a beneficial subset of cuboids to pre-compute using greedy benefit-cost optimization algorithms.

Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Draw and explain the 3-Tier Architecture of a Data Warehouse. (10 Marks)

Answer: 1. **Bottom Tier (Warehouse Database Server):** The relational database system holding the operational data extracted, transformed, and loaded via ETL tools (e.g., Informatica, DataStage). Stores metadata repository and data marts.

2. **Middle Tier (OLAP Server):** Implemented using ROLAP (relational engine with extended SQL joins) or MOLAP (multidimensional array storage engine) to execute multidimensional slice, dice, roll-up, and drill-down calculations.

3. **Top Tier (Front-End Client Tools):** Query and reporting tools, analysis tools, data mining algorithms, and executive dashboards.

Q2. Explain Bitmap Indexing and Join Indexing for OLAP query optimization. (8 Marks)

Bitmap Indexing: Used for attributes with low cardinality (few distinct values, e.g., `Gender` or `Marital\_Status`). A bit vector is maintained for each distinct value where bit $k = 1$ if tuple k contains the value, 0 otherwise. Logical AND/OR/NOT bitwise operations execute in single CPU clock cycles.

Join Indexing: Pre-computes relationships between foreign keys in fact tables and primary keys in dimension tables, eliminating runtime Cartesian join overhead during analytical SQL execution.

Q3. What is an Iceberg Cube and how does BUC (Bottom-Up Computation) prune search space? (6 Marks)

Answer: An **Iceberg Cube** computes only those cuboid cells whose aggregate measure satisfies a user-specified minimum threshold (e.g., $\text{count} \geq \text{min_sup}$). In BUC, cuboids are processed bottom-up from apex downwards. If an aggregate cell count falls below min_sup , the Apriori downward closure property guarantees that none of its specialized descendant sub-cells can meet the threshold, allowing BUC to immediately prune the entire subtree.

Q4. Compare Enterprise Warehouse, Data Mart, and Virtual Warehouse. (6 Marks)

Enterprise Warehouse: Collects all information about subjects spanning the entire organization. Provides corporate-wide data integration across all business units.

Data Mart: Contains a subset of corporate-wide data that is of value to a specific department or group of users (e.g., Marketing Data Mart).

Virtual Warehouse: A set of views over operational databases. Does not require separate physical storage; easy to build but imposes high query load on operational databases.